

# Programación y Algoritmos I – Tarea 7

Pedro D. Llerenas

pedro.llerenas@cimat.mx

2025-10-22

En este reporte, buscamos implementar un programa que reciba una cadena de caracteres y determine si esta tiene el formato de **notación científica**. En caso de que no sea así, el programa realizará la conversión correspondiente a su forma en notación científica.

La **notación científica** se utiliza para expresar números demasiado *grandes* (o *pequeños*) que resultan inconvenientes de escribir en forma decimal. Generalmente, esta notación emplea la base 10 para dichas representaciones. Por ejemplo:

| Decimal | Notación Científica  |
|---------|----------------------|
| 100     | $1 \times 10^2$      |
| 0.0003  | $3 \times 10^{-4}$   |
| 12.345  | $1.2345 \times 10^1$ |

Como primer solución, utilizaremos **expresiones regulares**. Las expresiones regulares consisten de constantes, que denotan conjuntos de strings, y operadores, que actúan sobre estos conjuntos de strings. Para nuestro objetivo, utilizaremos la siguiente expresión:

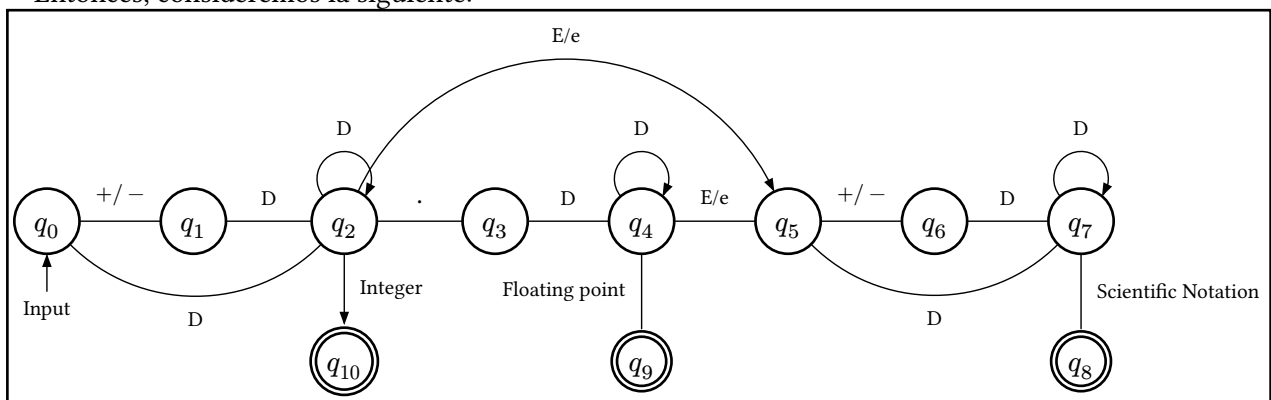
$[+-]?\d+(\. \d+)?([eE][+-]?\d+)$

En order, explicamos cada uno de los componentes:

| Expresión   | Descripción                                                                                    |
|-------------|------------------------------------------------------------------------------------------------|
| $[+-]?$     | Busca 0 o 1 ocurrencias de alguno de los caracteres '+' o '-'.                                 |
| $\d+$       | Busca una o más ocurrencias de un dígito (0-9)                                                 |
| $(\. \d+)?$ | Busca 0 o 1 ocurrencias de la combinación de un '.' y una o más ocurrencias de un dígito (0-9) |
| $[eE]$      | Busca una ocurrencia de 'e' o 'E'                                                              |

Nótese que los parentesis se utilizan para agrupar expresiones. De esta manera, podemos marcar como opcionales un conjunto de caracteres, pero si se presentan, deben aparecer de la manera descrita dentro de los parentesis.

La siguiente forma de resolver este problema, con la utilidad añadida de covertir/corregir ciertas expresiones a una notación científica normalizada, se hace a partir de una máquina de estado finito. Entonces, consideremos la siguiente:



Este diagrama se puede explicar de la siguiente manera:

| Nodo     | Descripción                                                                                                                                                                                                                      |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $q_0$    | Recibe string a analizar. Se dirige a $q_1$ si el primer caracter es +/-, a $q_2$ si es un dígito.                                                                                                                               |
| $q_1$    | Si el siguiente caracter del string es entero, avanzar a $q_2$                                                                                                                                                                   |
| $q_2$    | Realiza un bucle hasta que se dejen de encontrar dígitos contiguos. Si ya no hay más caracteres, el dígito es un entero, y se dirige a $q_{10}$ . Si el caracter es e/E, avanza a $q_5$ . Si es un punto decimal, avanza a $q_3$ |
| $q_3$    | El caracter debe ser un dígito. Si lo es, avanza a $q_4$                                                                                                                                                                         |
| $q_4$    | Un bucle que encuentre dígitos hasta que se acaben los caracteres (punto flotante), avanza a $q_9$ , o se encuentre una e/E, se avanza a $q_5$                                                                                   |
| $q_5$    | Busca +/- o un dígito. Si encuentra un signo, avanza a $q_6$ . Si encuentra un dígito, avanza a $q_7$                                                                                                                            |
| $q_6$    | Busca un dígito. Si lo es, avanza a $q_7$                                                                                                                                                                                        |
| $q_7$    | Bucle que encuentra dígitos, hasta que se terminen los caracteres. Avanza a $q_8$                                                                                                                                                |
| $q_8$    | Regresa que el string es notación científica                                                                                                                                                                                     |
| $q_9$    | Regresa que el string es de punto flotante                                                                                                                                                                                       |
| $q_{10}$ | Regresa que el string es un entero                                                                                                                                                                                               |

En particular, esta nos permite ver qué tipo de expresión se leyó: entera, de punto flotante, o en notación científica. Notemos que este sistema clasifica expresiones como 11.1e10 como notación científica. Nosotros buscamos normalizar esta representación.

El siguiente programa utiliza ambas técnicas para validar e intenta corregir las expresiones que no se encuentran en notación científica. Se compila usando

```
> g++ main.cpp -o main
```

Y se ejecuta usando

```
> ./main
```

Un ejemplo de interacción con el programa es el siguiente:

```
> =====
> Enter a number in scientific notation:
> 12.2
> Regex says: Invalid.
> Finite state machine says: 1.22e1
> =====
> =====
> Enter a number in scientific notation:
> 1.22e1
> Regex says: Valid.
> Finite state machine says: 1.22e1
> =====
> =====
> Enter a number in scientific notation:
> +12.2
```

```
> Regex says: Invalid.  
> Finite state machine says: +1.22e1  
> =====  
> =====  
> Enter a number in scientific notation:  
> +1.22e1  
> Regex says: Valid.  
> Finite state machine says: +1.22e1  
> =====  
> =====  
> Enter a number in scientific notation:  
> -0.023e-2  
> Regex says: Valid.  
> Finite state machine says: -0.023e-2  
> =====  
> =====  
> Enter a number in scientific notation:  
> -23.23e-3  
> Regex says: Valid.  
> Finite state machine says: -2.323e-2  
> =====
```

Notemos que el regex nos dice si la notación es válida o no, mientras que la máquina de estado finito nos normaliza la expresión.